

Database Design and ERD Creation

CE-UEBA | Context-Enriched User and Entity Behavior Analytics

CE-UEBA is an academic information-system proof of concept for the fictional digital bank FinansTech Ltd. Its relational database connects employee identity, IT assets, approved changes, behavioral events and risk alerts. This design supports explainable alert prioritization and suppression by combining an anomaly score with organizational context.

1 Key data entities

Entity and primary key	Purpose and principal attributes
roles / role_id	Reusable IAM role definitions: role_name and clearance_level.
users / user_id	Employee identity: username, full_name, email, department, employment_status, current and previous roles, and role_change_date.
it_assets / asset_id	IT inventory: hostname, ip_address, criticality_level, asset_type and owner_department.
change_tickets / ticket_id	Authorized change context: user and asset references, change_type, approved_start, approved_end, status and description.
behavioral_logs / log_id	Observed events: user and asset references, event_type, timestamp, details, source_ip and anomaly_type.
alerts / alert_id	Risk decisions: log and user references, raw_ml_risk_score, context_adjusted_risk_score, suppression_status, suppression_reason, severity, created_at, analyst_status and context_factors.

2 Diagram and integrity rules

The accompanying **CE_UEBA_ERD.png** contains all 45 attributes across the six entities, declared SQL types, primary keys (PK), foreign keys (FK), unique constraints (UQ), and nullability. Its relationship panel lists all eight foreign keys. The editable vector version is **CE_UEBA_ERD.svg**.

Each entity has an integer primary key. Role names, usernames, email addresses, hostnames and asset IP addresses are unique. Both user role references are optional; all other foreign-key columns are required. SQLite foreign-key enforcement is enabled for application connections. Role and asset records are separated from activity records to reduce repeated descriptive data.

Tool and implementation: The ERD was generated by a custom Python schema renderer using SQLite metadata and Pillow, with PNG and SVG output. The implemented database is SQLite through Flask-SQLAlchemy; this is a physical schema diagram, not a MySQL-specific model.

3 Relationships and cardinalities

All eight stored relationships are one-to-many from parent to child. A parent may have zero or many children. Each child references exactly one parent, except the two optional role references, which allow zero or one.

Parent key	Child foreign key	Parent to children
roles.role_id	users.role_id	1 : 0..N
roles.role_id	users.previous_role_id	1 : 0..N
users.user_id	change_tickets.user_id	1 : 0..N
it_assets.asset_id	change_tickets.asset_id	1 : 0..N
users.user_id	behavioral_logs.user_id	1 : 0..N
it_assets.asset_id	behavioral_logs.asset_id	1 : 0..N
users.user_id	alerts.user_id	1 : 0..N
behavioral_logs.log_id	alerts.log_id	1 : 0..N

One-to-one: No one-to-one relationship is enforced by the current database. The ORM exposes one alert per log through `uselist=False`, but `alerts.log_id` is not unique. If a strict maximum of one alert per log is required, a `UNIQUE` constraint on `alerts.log_id` would enforce log-to-alert 1 : 0..1; this is a proposed improvement, not an existing constraint.

Many-to-many: Users can interact with many assets, and an asset can be used by many users. Behavioral logs represent this association through two one-to-many relationships while storing each event. Change tickets likewise associate users and assets for change work. Repeated user-asset pairs are allowed; neither table is an access-permission mapping.

4 How the design supports functionality

For each behavioral event, the system retrieves the employee and current/previous role to assess employment status, clearance and recent role changes. The related asset provides criticality and departmental ownership. Change tickets are matched by user, asset, eligible status and event time within the approved window; there is no direct ticket-to-alert foreign key.

The risk engine adjusts the raw anomaly score using this context and records the result, severity, suppression decision and explanation in an alert. Retaining raw events separately from decisions supports drill-down review. Analyst status supports the review workflow, while user and asset joins support contextual dashboard views.

Design limits: `context_factors` is JSON-encoded text. The `users` table stores one previous role rather than a full role-history ledger. `alerts.user_id` duplicates the user reachable through its log; the foreign keys alone do not enforce equality between those two user references, so application logic must keep them consistent.

Validation sources: `models.py`; `instance/ce_ueba.db` table, index and foreign-key metadata; `risk_engine.py` ticket matching and risk evaluation. The model contains six tables, 45 columns and eight foreign keys.